

Aula 09 · Exercício: qual método é chamado?

Disciplina: Paradigmas de Linguagens de Programação

Tema: Tipos Abstratos de Dados e Orientação a Objetos (Sebesta, Caps. 11 e 12)

Aluno: Gabriel Santos Furlanette

Para cada trecho: **(a)** qual é a saída? **(b)** a decisão foi tomada na **compilação** ou na **execução**?

Exercício 1/3

Trecho 1 · Java

```
class Animal {
    String som() { return "..."; }
}
class Cachorro extends Animal {
    String som() { return "au"; }
}
Animal x = new Cachorro();
System.out.println(x.som());
```

- **(a) Saída:** au
- **(b) Execução.** Em Java, todo método de instância é vinculado dinamicamente (exceto `static`, `private` e `final`). A variável é do tipo `Animal`, mas o objeto é um `Cachorro`, então a JVM consulta a vtable do objeto e chama `Cachorro.som()`.

Trecho 2 · C++

```
struct A {
    void f() { cout << "A"; }
};
struct B : A {
    void f() { cout << "B"; }
};
B b; A *p = &b;
p->f();
```

- **(a) Saída:** A
 - **(b) Compilação.** Em C++ só métodos `virtual` são dinâmicos. Como `f()` não é `virtual`, o compilador escolhe o método pelo **tipo do ponteiro** (`A*`), e não pelo tipo do objeto. Se fosse `virtual void f()`, a saída seria B.
-

Exercício 2/3

Trecho 3 · Java

```
class A { String nome = "A";  
    String getNome() { return nome; } }  
class B extends A { String nome = "B";  
    String getNome() { return nome; } }  
A x = new B();  
System.out.println(x.nome + " " + x.getNome());
```

- **(a) Saída:** A B
- **(b) As duas coisas:**
- `x.nome` → **compilação**. Atributos não são polimórficos em Java: o campo é escolhido pelo tipo declarado (A). O campo `nome` de B apenas **oculta** o de A (os dois existem no objeto).
- `x.getNome()` → **execução**. O método é sobrescrito, então roda `B.getNome()`, que lê o `nome` de B.

Trecho 4 · Python

```
class Contador:  
    total = 0  
    def __init__(self):  
        Contador.total += 1  
        self.id = Contador.total  
  
a = Contador(); b = Contador()  
print(a.id, b.id, a.total)
```

- **(a) Saída:** 1 2 2
- **(b) Execução.** Em Python a busca de atributos é sempre feita em tempo de execução: primeiro no objeto, depois na classe. `total` é um atributo **de classe** (compartilhado), incrementado a cada construção. `a.total` não existe na instância `a`, então o Python encontra `Contador.total`, que já vale 2.

Exercício 3/3

Trecho 5 · Java

```

class A {
    static String quem() { return "A"; }
}
class B extends A {
    static String quem() { return "B"; }
}
A x = new B();
System.out.println(x.quem());

```

- **(a) Saída:** A
- **(b) Compilação.** Métodos `static` não são sobrescritos, só **ocultados**. A chamada é resolvida pelo tipo declarado da variável (A), e o objeto real é ignorado (equivale a `A.quem()`; o compilador até avisa que o certo é chamar pela classe).

Trecho 6 · Go

```

type Animal struct{}
func (Animal) Som() string { return "..."}
func (a Animal) Falar() string {
    return "faz " + a.Som()
}
type Cao struct{ Animal }
func (Cao) Som() string { return "au"}

fmt.Println(Cao{}.Falar(), Cao{}.Som())

```

- **(a) Saída:** faz ... au
- **(b) Compilação.** Go não tem herança, e sim **composição (embedding)**. `Falar()` é promovido de `Animal`, mas o receptor dentro dele é o `Animal` embutido, então `a.Som()` chama `Animal.Som()` → "...". Já `Cao{}.Som()` chama direto o método de `Cao` → "au". Não há vtable entre structs; despacho dinâmico em Go só acontece através de **interfaces**.

Resumo

#	Linguagem	Saída	Decisão
1	Java	au	Execução
2	C++	A	Compilação (sem <code>virtual</code>)
3	Java	A B	Campo: compilação · Método: execução
4	Python	1 2 2	Execução
5	Java	A	Compilação (<code>static</code>)
6	Go	faz ... au	Compilação (embedding ≠ herança)

Conclusão: só há despacho dinâmico quando a linguagem consulta o **tipo do objeto** em execução (métodos de instância em Java, `virtual` em C++, tudo em Python, interfaces em Go). Campos, métodos `static`, métodos não-virtuais e métodos promovidos por composição são resolvidos pelo **tipo declarado**, na compilação.